

COMMIT CONF 2026

Your queries are slow and (probably) it's *your* fault

Gonzalo Ortiz · StarTree Inc
Apache Pinot PMC member



slides

Look at this query

```
SELECT l_orderkey, l_extendedprice
FROM lineitem
WHERE YEAR(l_ship_ts) = 1995
      AND MONTH(l_ship_ts) = 3;
```

Looks normal, right? You've written this a hundred times.

Look at this query

```
SELECT l_orderkey, l_extendedprice
FROM lineitem
WHERE YEAR(l_ship_ts) = 1995
      AND MONTH(l_ship_ts) = 3;
```

Looks normal, right? You've written this a hundred times.

MEASURED · TPC-H SF=100 · 2 SERVERS

300 / 300 segments scanned
608M rows scanned in the filter
1.3 s wall-clock

The whole table lights up — to return 7.7M rows.

The "equivalent" rewrite

```
SELECT l_orderkey, l_extendedprice
FROM lineitem
WHERE l_ship_ts >= TIMESTAMP '1995-03-01'
      AND l_ship_ts <  TIMESTAMP '1995-04-01';
```

Same rows. Same result.

Its plan — same 7.7M rows

	YEAR(...)	raw range	
segments scanned	300	4	296 pruned
rows scanned in filter	608,915,652	0	609M → 0
time (24-core box)	1.3 s	0.7 s	work » wall-clock

The optimizer isn't dumb, but it short-sighted.

You blindfolded it — without realizing.

SETUP

The cast & the data

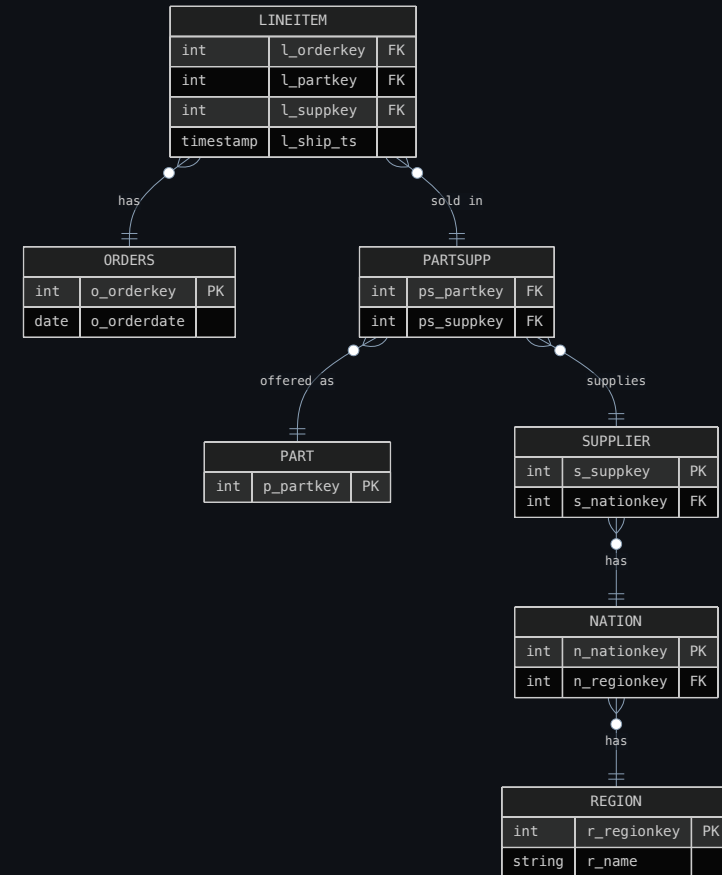
30 seconds on the engine, the field, and the dataset.

TPC-H — the dataset

A wholesale supplier's orders — the classic decision-support benchmark

SF=100

- **lineitem** $\approx$ 600M rows (~100 GB)
- **orders** 150M
- **partsupp** 80M
- **part** 20M
- **supplier** 1M



Apache Pinot — the reference engine

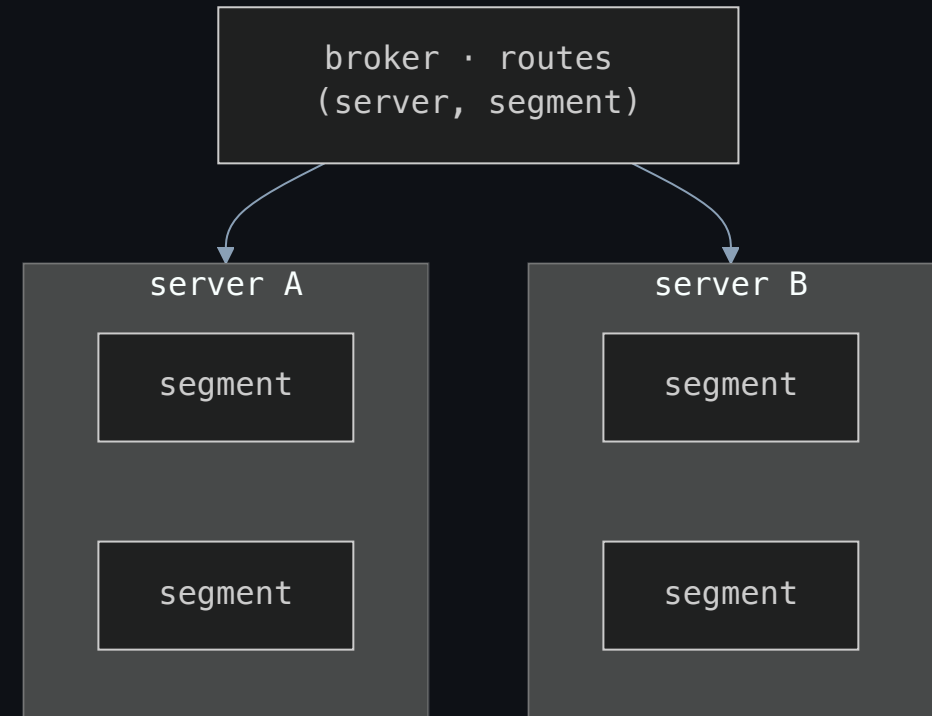
Real-time distributed OLAP — a relatively new category

Ultra-low latency, high QPS — <100 ms p99, 10k+ q/s, on PB-scale data

Columnar + rich indexes — inverted, range, star-tree, pre-aggregation

Scales out — no upper bound

User-facing analytics — dashboards, anomaly detection, ad-hoc



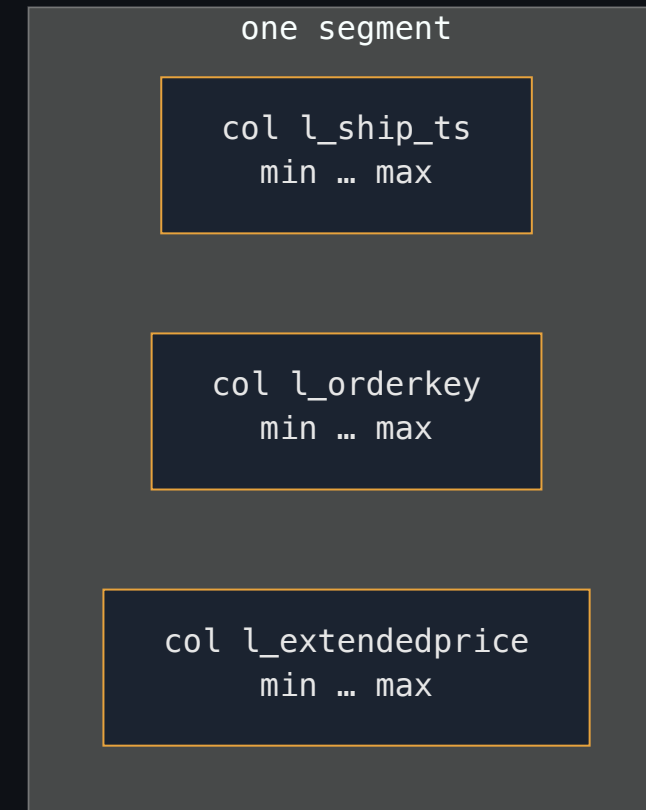
The segment — Pinot's atomic unit

The atomic unit — an immutable shard of the table

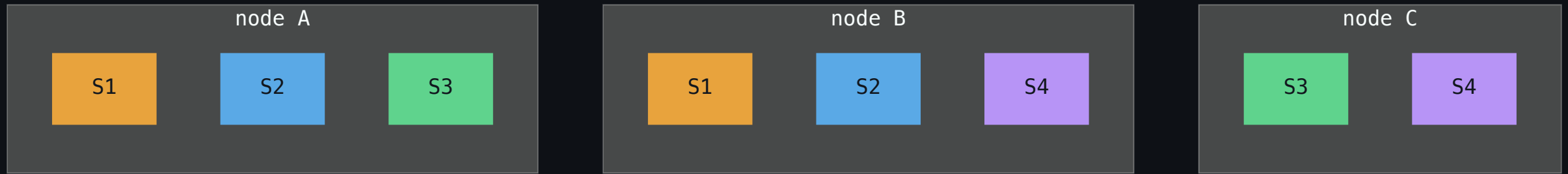
Columnar — stored and compressed column-by-column

Carries metadata — per-column min & max, plus indexes

Distributed across servers — and can be partitioned / co-located



Segments are distributed across servers



Usually the replication factor is 2

The other databases we'll name

PostgreSQL · MySQL · Oracle	classic row-store RDBMS (OLTP)
MongoDB	document store
ClickHouse	columnar OLAP, the closest to Pinot

TRAP #1

Columns you hide behind expressions

Remember the query from the very beginning? Time to explain it.

The query, explained — full scan

```
1 SELECT l_orderkey, l_extendedprice
2 FROM lineitem
3 WHERE YEAR(l_ship_ts) = 1995
4 AND MONTH(l_ship_ts) = 3;
```

PLAN + COST

```
filter on YEAR(l_ship_ts) → full scan
300 / 300 segments · 0 pruned
609M rows in filter · 1.3 s
```

1994-09

1994-11

1995-01

1995-03

1995-05

1995-07

Even if data is sorted, the optimizer doesn't know it can prune

The same answer — pruned

```
1 SELECT l_orderkey, l_extendedprice
2 FROM lineitem
3 WHERE l_ship_ts >= TIMESTAMP '1995-03-01'
4 AND l_ship_ts < TIMESTAMP '1995-04-01';
```

PLAN + COST

```
range on raw l_ship_ts → minmax/index prune
4 / 300 segments · 296 pruned
0 rows in filter · 0.7 s
```

x 1994-09

x 1994-11

x 1995-01

✓ 1995-03

x 1995-05

x 1995-07

A range on the *raw* column permits the optimization

Is this just Pinot? **No.**

<code>WHERE YEAR(col) = 1995</code>	uses index / prunes?
PostgreSQL	✗ seq scan
MongoDB	✗ COLLSCAN
MySQL	✗ full scan
Oracle	✗ full scan
Apache Pinot	✗ no prune
ClickHouse	✓ ...with an asterisk

The range always works.

TAKE-AWAY

Keep the column raw — transform the *literal*, not the column.
Functions may the pruner.

TRAP #2

The join that wanted to be a semi-join

A uniqueness invariant the optimizer can't see.

A join used only to *filter*

"Line items of parts supplied from Germany" — you join `partsupp` just to filter, and project only `lineitem`:

```
SELECT l.l_orderkey, l.l_extendedprice  -- only lineitem
FROM lineitem l                        -- 600M
  JOIN partsupp ps ON l.l_partkey = ps.ps_partkey  -- 80M
  JOIN supplier s  ON ps.ps_suppkey = s.s_suppkey  -- 1M
  JOIN nation n    ON s.s_nationkey = n.n_nationkey -- 25 units
WHERE n.n_name = 'GERMANY';
```

You only want line items that *have* a German supplier — a yes/no question.
But the DB needs to **scan the 600M lineitems** to do so!

Wouldn't it be nice to...

Wouldn't it be nice to...

```
SELECT n.n_nationkey FROM nation n
WHERE n.n_name = 'GERMANY';           -- 1 row
```

Wouldn't it be nice to...

```
SELECT n.n_nationkey FROM nation n
WHERE n.n_name = 'GERMANY';           -- 1 row
```

```
SELECT s.s_suppkey FROM supplier s
WHERE s.s_nationkey = <prev>;         -- uses inv index
```

Wouldn't it be nice to...

```
SELECT n.n_nationkey FROM nation n
WHERE n.n_name = 'GERMANY';           -- 1 row
```

```
SELECT s.s_suppkey FROM supplier s
WHERE s.s_nationkey = <prev>;         -- uses inv index
```

```
SELECT ps.ps_partkey FROM partsupp ps
WHERE ps.ps_suppkey IN <prev>;        -- uses inv index
```

Wouldn't it be nice to...

```
SELECT n.n_nationkey FROM nation n
WHERE n.n_name = 'GERMANY';           -- 1 row
```

```
SELECT s.s_suppkey FROM supplier s
WHERE s.s_nationkey = <prev>;         -- uses inv index
```

```
SELECT ps.ps_partkey FROM partsupp ps
WHERE ps.ps_suppkey IN <prev>;        -- uses inv index
```

```
SELECT l.l_orderkey, l.l_extendedprice FROM lineitem l
WHERE l.l_partkey IN <prev>;          -- uses inv index
```

Wouldn't it be nice to...

```
SELECT n.n_nationkey FROM nation n
WHERE n.n_name = 'GERMANY';           -- 1 row
```

```
SELECT s.s_suppkey FROM supplier s
WHERE s.s_nationkey = <prev>;         -- uses inv index
```

```
SELECT ps.ps_partkey FROM partsupp ps
WHERE ps.ps_suppkey IN <prev>;        -- uses inv index
```

```
SELECT l.l_orderkey, l.l_extendedprice FROM lineitem l
WHERE l.l_partkey IN <prev>;          -- uses inv index
```

That's exactly a **dynamic filter** — Trino & AnalyticDB do it; Pinot, partially.

What about Pinot?

Has it for semi-joins — `... WHERE x IN (SELECT ...)`

Join → semi-join — only when the right side is known **unique**

Why?

- A right side with repetitions makes a *join* duplicate rows — a semi-join won't
- The semi-join is probably what you meant
- But the optimizer can't assume it

Give it the proof: `DISTINCT` (adds cost) · a **hint** · or just write the semi-join.

How does it work in Trino?

TAKE-AWAY

A join used only to filter wants to be a semi-join

- It lets the engine build the small side first and probe the big side *through its index*
- It is very DB-dependent

TRAP #3

Distributed joins

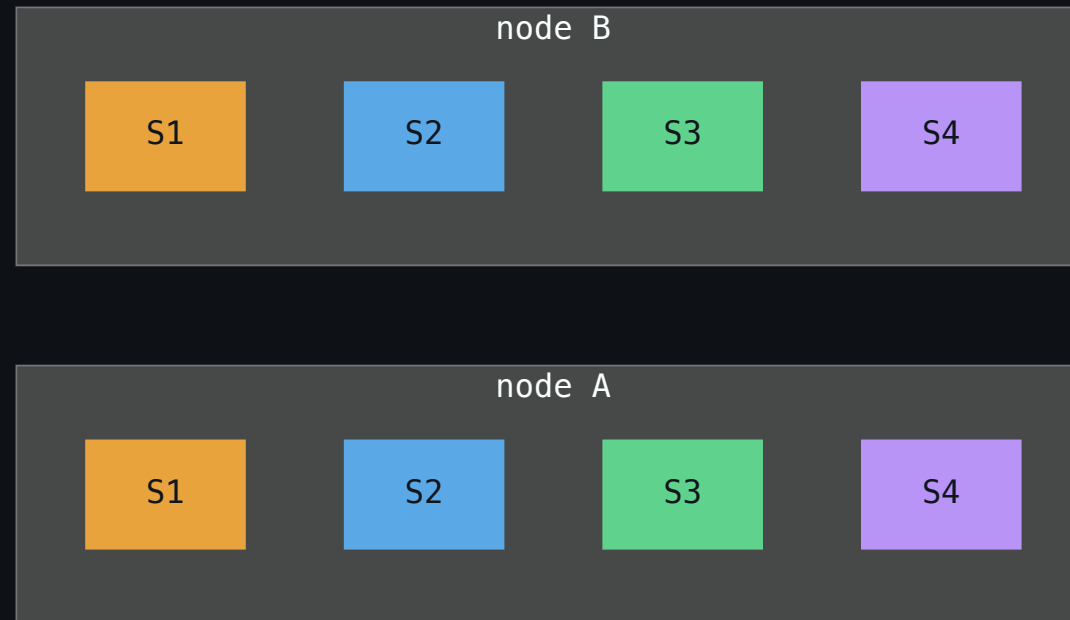
Where your data lives decides what a join costs.

Single instance



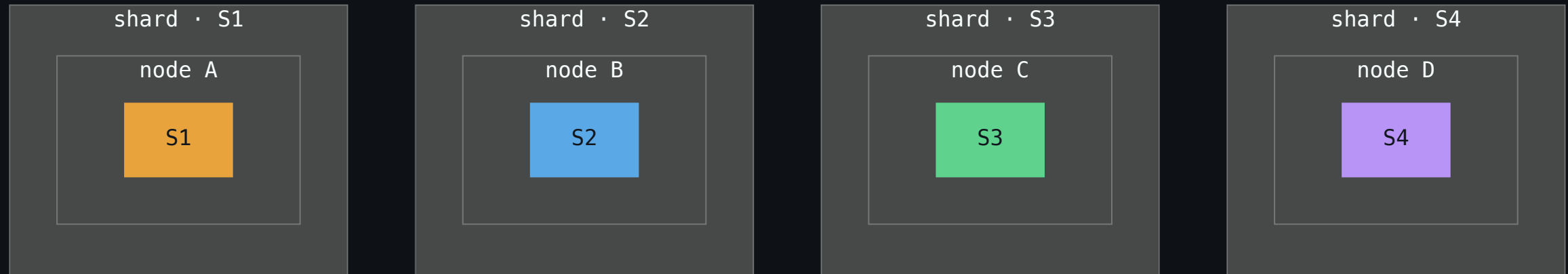
Every database.

Replication



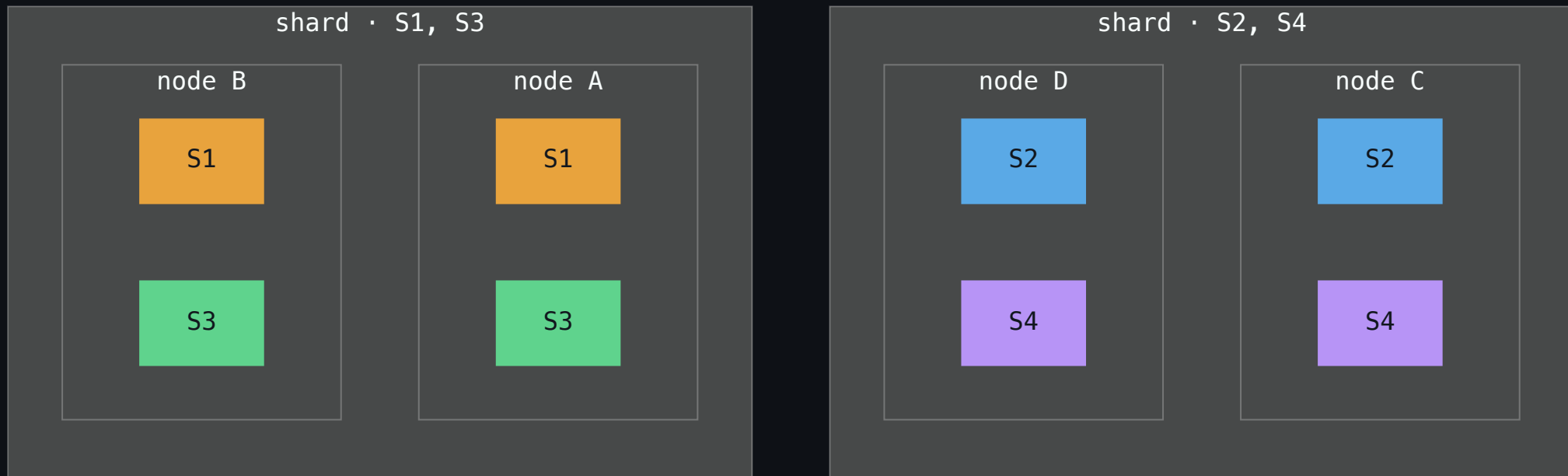
All of them — Postgres · MySQL · MongoDB · ClickHouse · Pinot.

Sharding



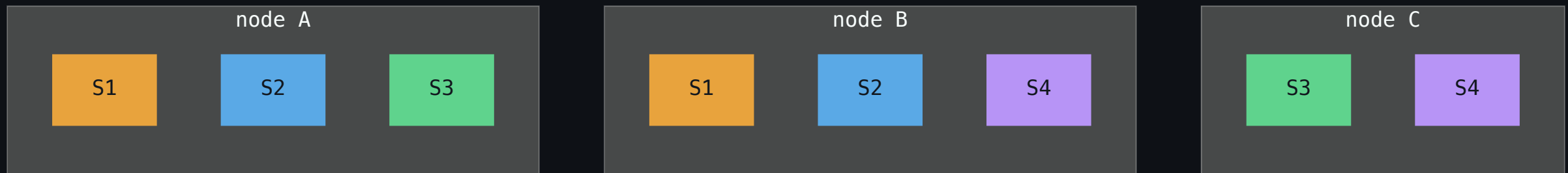
MongoDB · ClickHouse · Pinot natively · Postgres/MySQL via add-ons (Citus, Vitess).

Replication + sharding



MongoDB · ClickHouse · Pinot.

Pinot's default — segment-level



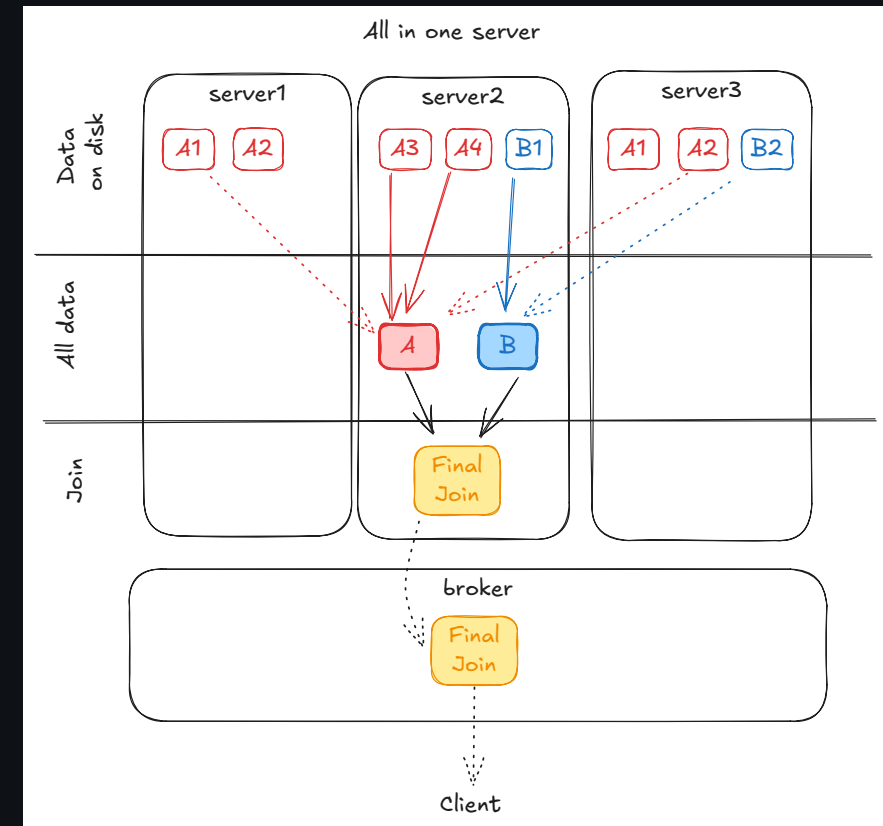
Pinot, by default — every segment on ≥ 2 nodes. Rare elsewhere.

Move everything to one server

the simplest distributed join: **move all the data to one server**

rows shuffled $\leq n(A) + n(B)$

not scalable — one server holds everything

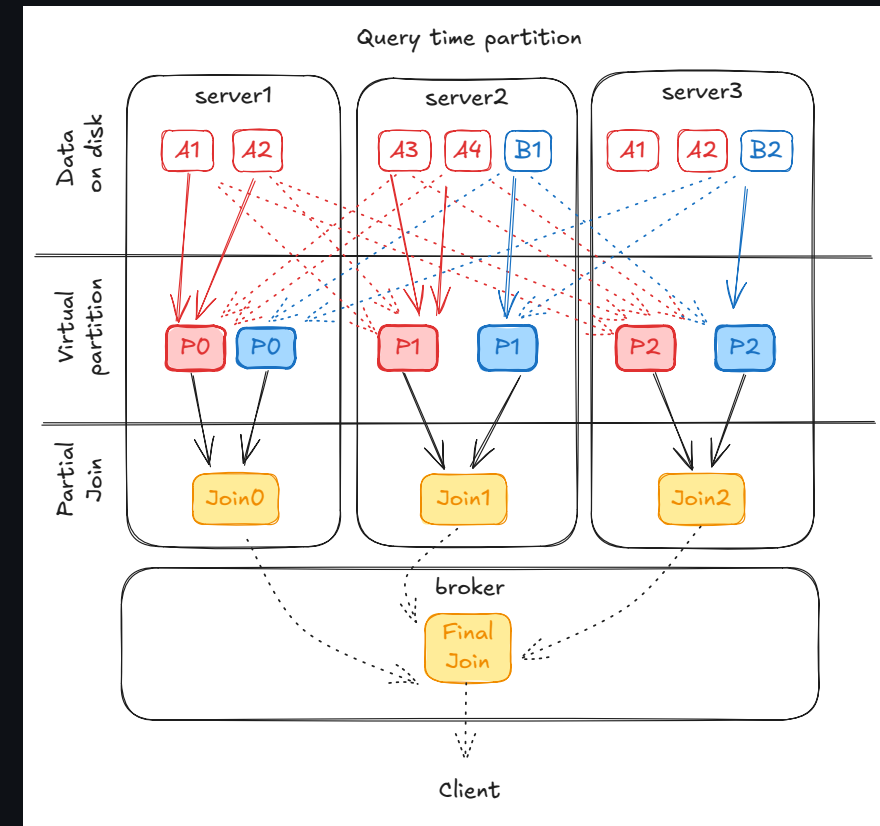


Query-time partitioning

for equi-joins (ON A.col2 = B.col3)

create virtual partitions by the join key at query time

rows shuffled $\leq n(A) + n(B)$

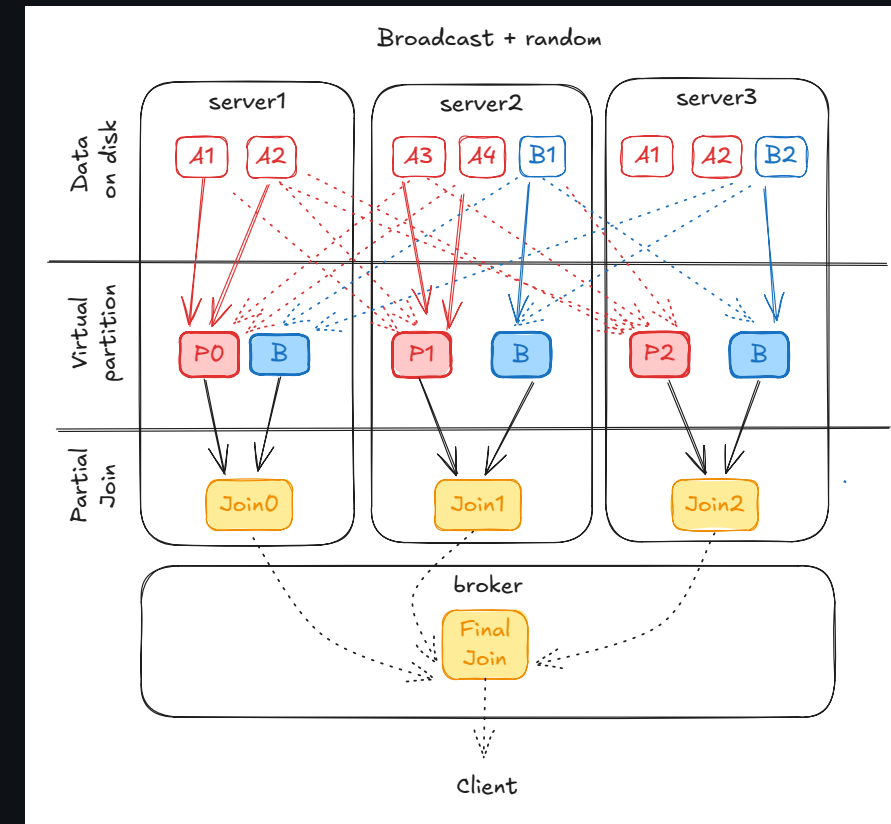


Broadcast + random

for non-equi joins (`ON A.col2 > B.col3`,
`!=`)

both sides can't be partitioned → copy the
small side everywhere

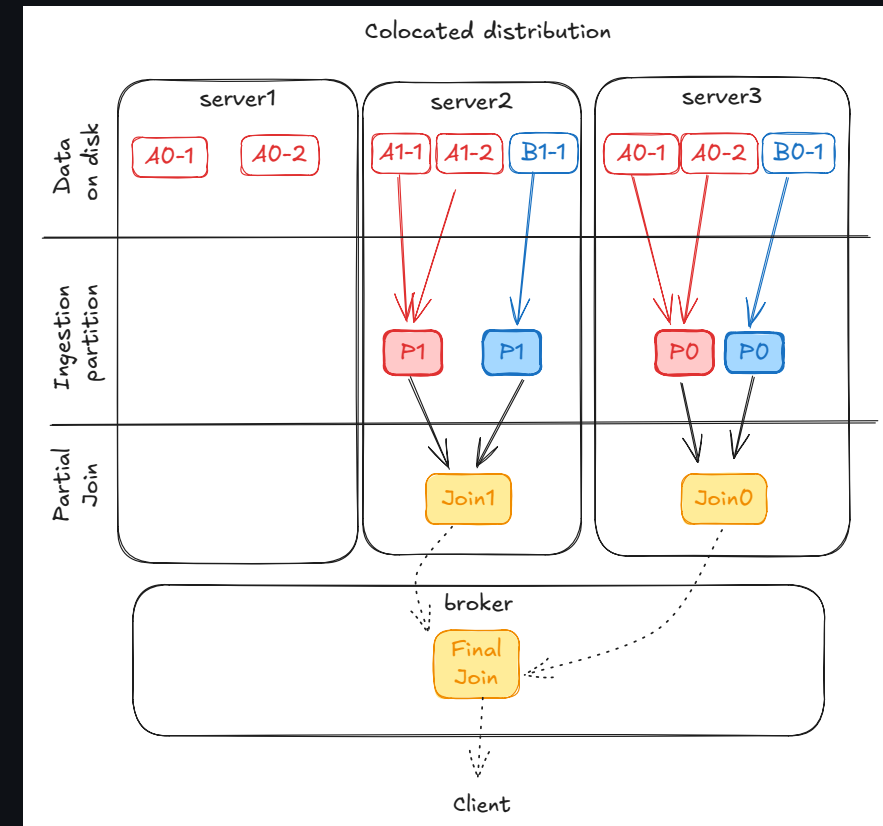
rows shuffled $\leq n(A) + n(B) \times \text{numServers}$



Co-located — nothing moves

for **equi-joins** where both tables are
partitioned on the join columns
and the partitions are **co-located** on the
same servers

0 data shuffle



TAKE-AWAY

In a distributed database, *where the data lives* is critical for the query plan.

TRAP #4

Paginating with OFFSET

Page 1 is slow. Page 1,000 is a disaster.

A perfectly reasonable pagination

One year of orders, newest first, 10 per page:

```
SELECT * FROM lineitem
WHERE l_ship_ts >= TIMESTAMP '1995-01-01'
      AND l_ship_ts <  TIMESTAMP '1996-01-01'
ORDER BY l_ship_ts DESC
LIMIT 10 OFFSET :page;
```

~41M rows in that window · 44 of 300 segments.

A perfectly reasonable pagination

One year of orders, newest first, 10 per page:

```
SELECT * FROM lineitem
WHERE l_ship_ts >= TIMESTAMP '1995-01-01'
      AND l_ship_ts <  TIMESTAMP '1996-01-01'
ORDER BY l_ship_ts DESC
LIMIT 10 OFFSET :page;
```

~41M rows in that window · 44 of 300 segments.

```
MEASURED · SF=100 · OFFSET 0 (PAGE 1)
44 / 300 segments
40,904,672 rows scanned
3.4 s
```

Even *page 1* reads all 41M rows to return 10 — the range prunes to the window, but the `ORDER BY ... LIMIT` segment-skip is disabled by the filter. (a real bug — two slides on.)

Deeper pages: same scan, exploding time

OFFSET	rows scanned	time
0	40.9M	3.4 s
100,000	40.9M	7.8 s
1,000,000	40.9M	29 s

same data scanned every time

8.6× slower — pure **OFFSET** tax

each server *produces, sorts, ships, then discards* OFFSET+LIMIT rows

Keyset pagination — carry a cursor

```
1 SELECT * FROM lineitem
2 WHERE l_ship_ts >= TIMESTAMP '1995-01-01'
3    AND (l_ship_ts, l_orderkey, l_linenum)
4         < (:last_ts, :last_orderkey, :last_line)
5 ORDER BY
6    l_ship_ts DESC,
7    l_orderkey DESC,
8    l_linenum DESC
9 LIMIT 10;
```

No offset tax — nothing produced just to be thrown away

The cursor shrinks the range — deeper pages prune *more* segments

Flat, not exploding — vs OFFSET's
3.4 → 29 s blow-up

In a DB with a sorted index, this is $O(\log n)$

In parallel queries OFFSET is even worse

Bonus: LIMIT without ORDER BY

Non-deterministic across runs — each server contributes whatever it likes.

TAKE-AWAY

Don't deep-paginate analytics. **OFFSET** makes every server do work it throws away — and a **WHERE** means it can't even skip.



use-the-index-luke.com · Markus Winand

BONUS

What about your ORM?

ORMs don't doom you — but they make it easy

You *can* write efficient queries — ORMs aren't the enemy

...but inefficiency is the easy path — the N+1 problem, cartesian fetch on collection joins

An extra layer that loses information — so the three traps creep in where you can't see

It must know your invariants too — types, keys, uniqueness

So...

Read the SQL it generates — Usually in logs

Or don't use it for queries — jOOQ is amazing for that

Turn logging on and go find your traps in the wild.

The checklist

1. Transform the **literal**, not the column.
2. Design your joins
 - Cardinality
 - Distribution
3. Keyset pagination, never **OFFSET** at depth.
4. **Read the plan** — it's free.
5. Turn on your ORM's SQL logging.

YOUR TAKE-HOME KIT

Audit your queries on Monday

The optimizer is your partner, not your enemy.

It just needs you to tell it the truth.

Gonzalo Ortiz · StarTree Inc
Apache Pinot PMC member
thanks 🙏

© 2026 Gonzalo Ortiz · slides licensed CC BY 4.0



slides